Query Scheduling Optimization in IVF-based Vector Search

Yujia Qian
yujia_qian@gsd.harvard.edu
Harvard University
Cambridge, Massachusetts, USA

Xiangyu Guan
xiangyu_guan@gsd.harvard.edu
Harvard University
Cambridge, Massachusetts, USA

Abstract

Vector search is a core component of modern AI systems, yet most optimization efforts focus on index structure rather than query execution. This paper investigates whether throughput can be improved by batching concurrent queries over an IVF index to exploit overlap in the inverted lists they probe, without modifying the index itself. We compare two batching strategies, Batch(MV) and Batch(MM), against a sequential baseline under random and clustered query workloads. Experiments on SIFT1M confirm that batching consistently improves throughput over sequential execution: Batch(MV) outperforms Sequential by up to 19% on random queries, while Batch(MM) achieves up to 40% gains on clustered queries, with all schedulers preserving identical recall. The preferred scan mode shifts with workload structure, and batch size further modulates the relative performance between MV and MM. A microbenchmark isolates the mechanism underlying these differences and supports practical guidance on applying batch scheduling in IVF-based vector search.

Keywords

vector search, IVF, batch scheduling, GEMM, query execution

1 Introduction

Vector search is central to a wide range of AI applications, including retrieval-augmented generation, semantic search, and recommendation systems. Given a query vector, the system must efficiently return the k nearest neighbors from a large corpus of stored vectors. At scale, such systems must sustain high query throughput while keeping tail latency bounded.

The dominant approach to improving vector search performance is to optimize the index structure. Hierarchical Navigable Small World graphs (HNSW) [2], product quantization, and IVF variants have each offered significant gains in recall-efficiency trade-offs. By contrast, the query execution layer—how individual queries are dispatched and processed at runtime—has received comparatively less attention.

This work takes the index as fixed and asks: **can throughput be improved by reorganizing how concurrent queries are executed?**

We focus on IVF (Inverted File Index), which partitions the vector space into clusters, each maintaining an inverted list of assigned vectors. To answer a query, the engine identifies the nearest centroids and scans the corresponding lists. Crucially, when multiple queries are processed together, many of them probe overlapping lists. If the engine exploits this overlap—loading each list once and computing distances for all queries that need it—the total number of memory loads decreases substantially.

To evaluate this approach, we design experiments under two workload conditions: random queries and clustered queries. We expect the degree of list overlap—and thus the benefit of batching—to differ substantially between the two, making them a natural test of when the approach succeeds and when it does not. We further consider two scan modes: Batch(MV), which computes distances for each query individually via matrix-vector multiplication, and Batch(MM), which stacks all queries sharing a list into a single matrix-matrix multiplication. We examine how the choice of scan mode and batch size affects throughput under each workload.

Our experiments reveal clear performance differences across workloads and scan modes, and offer practical insights into how batch scheduling strategies can be applied to improve throughput in IVF-based vector search.

2 Method

2.1 System Design

2.1.1 IVF Engine. Our engine follows the IVF design from Faiss [1]: base vectors are partitioned into 256 clusters via k-means, each cluster maintaining an inverted list. We use Faiss for centroid training but replace `index.search()` with a custom single-threaded implementation. Faiss's search is a stateless black box that processes each query independently, making cross-query optimizations invisible. Our engine exposes centroid lookup and list scanning as separate interfaces, giving the scheduler direct control over execution order and batch composition.

2.1.2 Per-List Batch Scan. When a batch of queries is dispatched together, multiple queries often probe the same inverted list. The per-list batch scan exploits this by iterating over inverted lists in list-major order: for each list, distances are computed for all queries that probe it before moving to the next. Each list is therefore loaded from memory exactly once per batch, regardless of how many queries share it.

We define **L** as the number of queries within a batch that probe the same inverted list. Under sequential execution, $L = 1$ always. Under time-window batching, L grows with batch size and workload locality, and directly determines which scan mode is more efficient.

2.1.3 Scan Modes. Two implementations of the per-list distance computation are provided, differing in how they handle the L queries sharing each list:

Batch(MV). Distances are computed for each of the L queries individually via a matrix-vector multiply (`vecs @ q`). Arithmetic intensity is constant at 0.5 FLOP/byte, independent of L.

Batch(MM). All L queries are stacked into a query matrix and distances are computed via a single matrix-matrix multiply (`vecs @ Q.T`). Arithmetic intensity scales as $L/2$ FLOP/byte, making MM increasingly efficient as L grows. However, GEMM carries nontrivial setup overhead at small L, degrading performance relative

Table 1: Scheduler configurations.

Scheduler	Scan mode	Batching policy
Sequential	Single-query GEMV per list	None
Batch(MV)	Per-query GEMV per list	Time + batch size
Batch(MM)	GEMM per list	Time + batch size

to MV. The crossover point is hardware-specific and is measured empirically in Section 3.3.

2.1.4 Schedulers. Three configurations are evaluated (Table 1).

Time-window batching accumulates arriving queries until either a time limit Δt elapses or a maximum batch size MaxBS is reached, then dispatches all accumulated queries as a single batch. The dual trigger bounds both waiting time at low load and memory pressure at high load.

2.2 Experiment Design

2.2.1 Dataset. All experiments use the SIFT1M benchmark: one million 128-dimensional SIFT descriptors, evaluated under L2 distance, with 10,000 held-out test queries and precomputed ground truth. All vectors reside in memory; no disk I/O occurs during search.

2.2.2 Workloads. Query arrivals are simulated at a target rate of 2,000 QPS. Two workloads are evaluated:

Random. All 10,000 SIFT test queries are used in their original order. Each query probes a largely disjoint set of 8 inverted lists, keeping L small. This represents an adversarial case for batching: list-reuse benefit is low, and any overhead from batch management directly reduces throughput.

Clustered. Queries are filtered to those whose nearest centroid falls among the 10 most-queried centroids, yielding 791 unique queries. These are sorted by primary centroid and then tiled (round-robin repetition) to reach 10,000 arrivals. Sorting by centroid ensures that consecutive arrivals within each time window belong to the same cluster, maximising L per list per batch. All 791 queries are real SIFT test vectors with valid ground truth, so Recall@10 is well-defined.

2.2.3 Device. All experiments are conducted on a MacBook Pro with an Apple M3 Pro chip and 18 GB unified memory. The engine runs single-threaded throughout to isolate scheduling effects from multi-core parallelism.

2.2.4 Experimental Protocol. Three experiments address the following research questions:

- **RQ1.** Does time-window batching improve throughput over sequential execution, under both random and clustered workloads?
- **RQ2.** How does scan mode choice (MV vs MM) affect performance under random versus clustered query distributions?
- **RQ3.** How does batch size affect the performance of batch schedulers?
- **RQ4.** What is the underlying mechanism that governs the relative performance of the two scan modes?

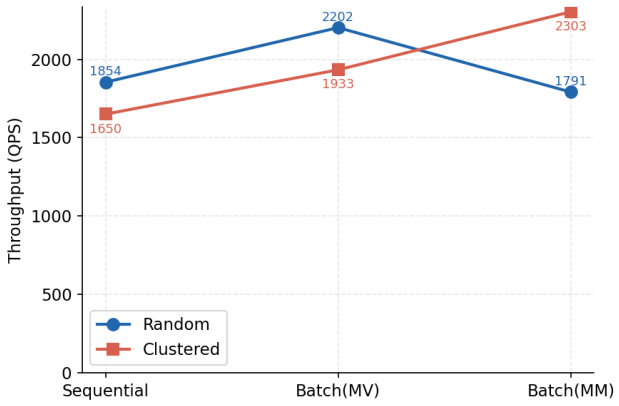

Figure 1: Throughput (QPS) across schedulers for random and clustered workloads.

Experiment 1 — Main Experiment (RQ1, RQ2). All three schedulers are evaluated on both workloads under fixed parameters ($n_{\text{clusters}} = 256$, nprobe = 8, $k = 10$, $\Delta t = 5$ ms, MaxBS = 128) across 5 independent runs. Summary statistics are reported as mean ± std; P99 latency is the median across runs.

Experiment 2 — Batch Size Parameter Sweep (RQ3). A grid search over $\Delta t \in \{0.5, 1, 2, 5, 10, 20, 50\}$ ms and MaxBS $\in \{32, 64, 128, 256\}$ is conducted once on both workloads.

Experiment 3 — Microbenchmark (RQ4). The MV and MM kernels are evaluated in isolation on a synthetic inverted list across a range of L values, with 50 repetitions per point.

3 Results

We present results in three stages. First, the main experiment demonstrates the core throughput claims under fixed scheduler parameters. Second, the batch size parameter sweep shows how batch size modulates the relative performance of MV and MM. Third, the microbenchmark isolates L as the underlying mechanism explaining both.

3.1 Main Experiment

All three schedulers are evaluated at fixed parameters ($\Delta t = 5$ ms, MaxBS = 128) across 5 independent runs.

3.1.1 Performance Overview. Figure 1 shows throughput across all three schedulers under both workloads. The two lines cross between Batch(MV) and Batch(MM): on the random workload, throughput peaks at Batch(MV) then falls for MM; on the clustered workload, it rises monotonically with MM taking the lead. This crossing is the central result of the paper.

Two findings stand out. First, batching consistently outperforms Sequential when the workload is favorable: Batch(MV) achieves +19% on random, and Batch(MM) achieves +40% on clustered. Second, the relative ordering of MV and MM is reversed across workloads—MV leads on random while MM leads on clustered. The throughput gains of batching come at the cost of higher per-query latency due to queueing: average latency rises from ∼0.5 ms (Sequential) to 58–105 ms for batch schedulers. Recall@10 is identical

Table 2: Throughput and Recall@10 (mean ± std, 5 runs). $\Delta t = 5$ ms, MaxBS = 128.

Scheduler	QPS	Avg (ms)	P95 (ms)	P99 (ms)	Recall @10
Random workload					
Sequential	1,854 ± 58	0.54	0.74	0.81	0.957
Batch(MV)	**2,202 ± 23**	74.34	106.79	114.67	0.957
Batch(MM)	1,791 ± 87	104.69	134.60	152.89	0.957
Clustered workload					
Sequential	1,650 ± 33	0.61	0.78	0.88	0.981
Batch(MV)	1,933 ± 39	96.19	127.30	139.06	0.981
Batch(MM)	**2,303 ± 111**	58.09	87.15	106.98	0.981

Table 3: Mean per-list reuse metrics across 5 runs.

Scheduler	Lists/query	L mean	L P50	L P95
Random workload				
Sequential	8.00	1.0	1	1
Batch(MV)	2.23	3.6	3	8
Batch(MM)	1.92	4.2	4	10
Clustered workload				
Sequential	8.00	1.0	1	1
Batch(MV)	0.55	14.6	5	64
Batch(MM)	0.65	12.4	5	52

across all schedulers within each workload, confirming that the gains arise from execution reorganization rather than approximation.

3.1.2 List Sharing (L). Sequential always loads 8 lists per query with $L = 1$. On the random workload, batching reduces unique list loads per query to 2.23 (3.6× reduction), with mean $L = 3.6$–4.2. On the clustered workload, this reduction reaches 0.55 lists per query, with mean $L = 12.4$–14.6. The contrast in L across the two workloads points to the underlying mechanism distinguishing MV and MM performance—addressed directly by the microbenchmark.

3.2 Batch Size Parameter Sweep

We sweep Δt and MaxBS on both workloads to examine how batch size affects the MV–MM trade-off. Four representative configurations are selected per workload. The sequential baseline achieves 1,937 QPS on random and 1,650 QPS on clustered. In Tables 4 and 5, ΔQPS is computed as (MM − MV)/MV.

3.2.1 Random Workload. As Δt and MaxBS grow, L increases and MM's disadvantage narrows and eventually reverses. At the smallest configuration ($\Delta t = 0.5$ ms, MaxBS = 32), $L \approx 1.5$–1.7 places both schedulers well below the hardware crossover, and MM's GEMM setup overhead dominates (−38%). At the main experiment parameters ($\Delta t = 5$ ms, MaxBS = 128), L rises to 3.1–3.5 and the gap nearly closes (−2%). At $\Delta t = 20$ ms and MaxBS = 256, L reaches 5.8–7.3 and MM overtakes MV by 10%. The throughput gains come at a direct latency cost: MV average latency grows from 17.5 ms to 160.8 ms, while the sequential baseline maintains ∼0.5 ms.

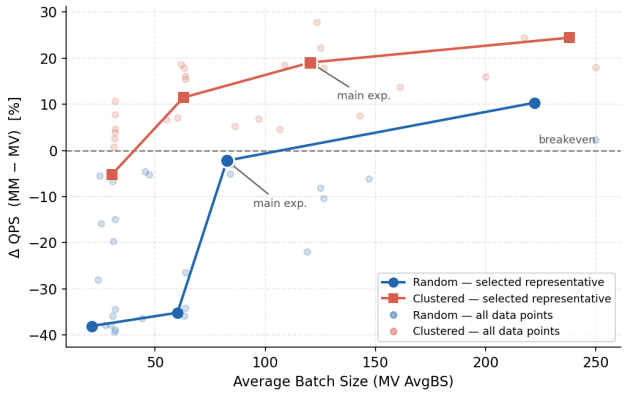

Figure 2: Batch size sweep: relative QPS of MM vs MV across random and clustered workloads.

3.2.2 Clustered Workload. In contrast to the random workload, MM wins from small-to-medium batch sizes because clustered queries concentrate on overlapping lists, keeping L above the crossover even in small batches. At $\Delta t = 0.5$ ms and MaxBS = 32, $L \approx 7.5$–7.6 sits right at the hardware crossover and MM barely loses (−5%). As batch size grows, L rises well above the crossover and MM's advantage increases steadily to +24%.

Figure 2 summarises the sweep results across both workloads. The two trajectories—random rising from −38% and clustered starting near zero—illustrate how workload structure determines the operating regime of each scan mode.

Figures 3 and 4 show the latency cost of batching. On the random workload, both schedulers incur substantially higher average latency than Sequential (∼0.5 ms); at small batches MM latency exceeds MV's due to GEMM setup overhead, but at large batches MM's faster execution drains the queue sooner. On the clustered workload, MM latency runs consistently below MV because the high list-sharing rate gives MM a sustained throughput advantage.

3.3 Microbenchmark

To isolate the effect of L from scheduler dynamics, the MV and MM kernels are benchmarked directly on a synthetic inverted list ($n = 4{,}000$ vectors, $d = 128$) for $L \in \{1, \ldots, 256\}$, with 50 repetitions per point. Figure 5 reports kernel cost in ns/(query × vector), so lower values indicate higher throughput.

MV throughput is approximately constant across all values of L (5.1–5.5 ns/q·v), as each query is processed independently. MM throughput improves sharply with L, crossing the MV baseline at $L = 8$ and reaching a 3.77× speedup at $L = 256$. The crossover corresponds to the point where GEMM arithmetic intensity ($L/2$ FLOP/byte) exceeds the GEMV baseline (0.5 FLOP/byte). In the main experiment, mean $L \approx 3.6$–4.2 on random (below the crossover) and mean $L \approx 12.4$–14.6 on clustered (above it), fully accounting for the reversed MV–MM ordering observed in Table 2.

Table 4: Selected (Δt, MaxBS) configurations — random workload.

Δt (ms)	MaxBS	MV QPS	MM QPS	Mean L (MV/MM)	MV Lat. (ms)	ΔQPS (MM−MV) (%)
0.5	32	2,098	1,300	1.5/1.7	17.5	−38%
5	64	2,230	1,446	2.5/2.5	42.3	−35%
5	**128**	**2,277**	**2,227**	**3.1/3.5**	**60.1**	−2%
20	256	2,322	2,562	7.3/5.8	160.8	+10%

Table 5: Selected (Δt, MaxBS) configurations — clustered workload.

Δt (ms)	MaxBS	MV QPS	MM QPS	Mean L (MV/MM)	MV Lat. (ms)	ΔQPS (MM−MV) (%)
0.5	32	1,987	1,884	7.5/7.6	23.2	−5%
5	64	1,934	2,156	11.1/11.0	48.7	+11%
5	**128**	**1,952**	**2,324**	**14.6/11.9**	**95.4**	**+19%**
20	256	1,948	2,424	17.8/17.3	192.3	+24%

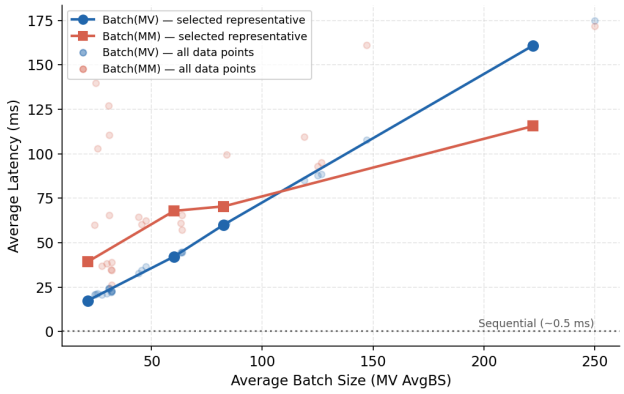

Figure 3: Average latency vs batch size on the random workload.

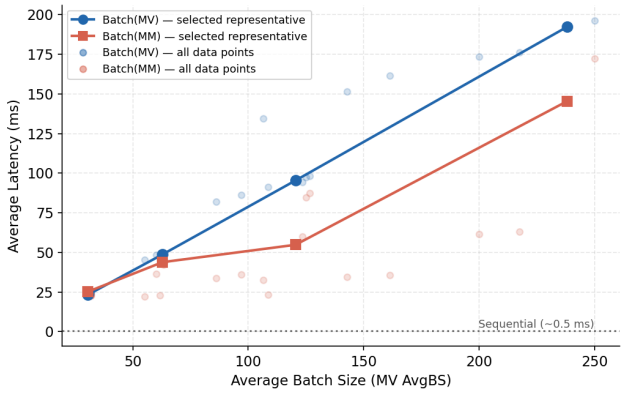

Figure 4: Average latency vs batch size on the clustered workload.

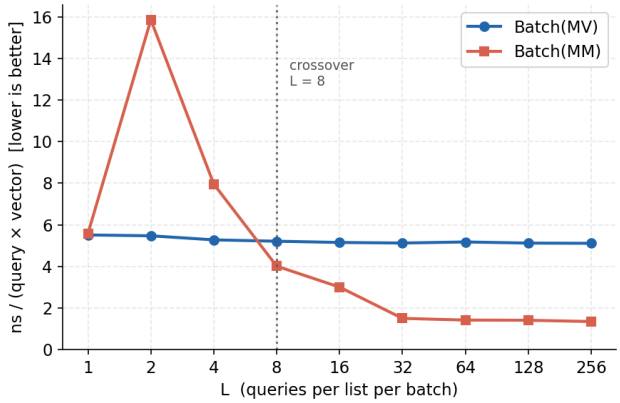

Figure 5: Scan kernel throughput as a function of L.

4 Discussion

RQ1. Batching can meaningfully improve throughput over sequential execution, though the degree depends on both the scan mode and the workload. Batch(MV) outperforms Sequential on both workloads; Batch(MM) does so only on the clustered workload, where it achieves +40%. On the random workload, Batch(MM) falls 3% below Sequential—the queueing cost of batching is not recovered when per-batch scan overhead is high. The primary source of gain is list reuse: batching reduces unique list loads per query from 8 to 2.23 on random and 0.55 on clustered, a reduction of up to 14.5× relative to sequential processing.

RQ2. Query distribution substantially affects the ranking of all three schedulers. On the random workload, Batch(MV) is the clear winner while Batch(MM) falls below even Sequential; on the clustered workload, Batch(MM) takes the lead and Sequential is last. Neither batch mode dominates unconditionally—the workload determines which strategy is most effective.

RQ3. Larger batch sizes raise L across both workloads. On the random workload, this narrows the gap between MV and MM but does not invert it. On the clustered workload, MM's lead is robust

across all batch sizes. Batch size modulates the magnitude of the performance differences but does not alter their direction given a fixed query distribution.

RQ4. The microbenchmark isolates L as the governing variable. MV throughput is approximately constant regardless of L, while MM throughput improves sharply with L, crossing the MV baseline at $L = 8$. This threshold corresponds to the point at which GEMM arithmetic intensity ($L/2$ FLOP/byte) exceeds that of GEMV (0.5 FLOP/byte) on the experimental hardware. In the main experiment, mean $L \approx 3.6$–4.2 on random places Batch(MM) below this crossover, while mean $L \approx 12.4$–14.6 on clustered places it well above, fully accounting for the reversed ordering in Table 2.

Summary. Time-window batching consistently outperforms sequential execution without modifying the index or compromising recall. The key variable governing scan mode selection is L—the number of queries sharing each inverted list per batch. When L is below the crossover threshold (~8 on the experimental hardware), Batch(MV) is preferable; above it, Batch(MM) is preferable. L is shaped by *batch size* (jointly determined by target QPS and latency budget) and *query distribution* (which determines list overlap per batch). The gains, however, come at the cost of higher per-query latency due to queueing, and the throughput–latency trade-off must be weighed against the application's tail-latency requirements.

Limitations and Future Work. The engine is deliberately single-threaded to isolate scheduling effects; whether the observed gains survive in a multi-core deployment remains an open question. An adaptive strategy that switches between MV and MM per batch based on observed L would eliminate the regression on random workloads while preserving MM's advantage on clustered ones. All results are reported on SIFT1M; validation on datasets with different dimensionalities or cluster geometries would strengthen generalizability.

References

[1] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* (2021).

[2] Yuri A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2020).